

RESUMEIQ: RECRUITMENT DECISION-SUPPORT PLATFORM

Final Internship Technical Report · Architecture, NLP Pipeline & Empirical Evaluation

Author: Software Engineering Intern | Date: October 2026 | Repo: dan1a-ahm3d/resumeIQ | Commit: bd75757

1. INTRODUCTION

In modern technical recruitment, talent acquisition teams face overwhelming volumes of resumes for specialized engineering positions. Manual screening is labor-intensive, subject to fatigue-induced inconsistencies, and prone to subjective evaluation variances. While recent industry trends have turned toward generative Large Language Models (LLMs) to automate hiring workflows, generative approaches introduce severe operational and ethical risks: stochastic hallucinations, non-verifiable scoring criteria, lack of explainability, and uncalibrated demographic bias.

ResumeIQ was engineered to resolve these challenges by establishing an objective, explainable, and deterministic recruiter decision-support system. Rather than delegating employment determinations to opaque black-box models, ResumeIQ calculates verifiable lexical alignment metrics, extracts sentence-level evidence citations, and produces deterministic candidate rankings. Final hiring decisions and candidate evaluations remain strictly with human recruiters.

2. ABSTRACT

3. TOOLS USED

4. STEPS INVOLVED (ARCHITECTURE, INTEGRATION & TUNING)

- Step 1: Design System & Frontend Architecture (Milestone 1)
Extracted ergonomic tokens from the Stitch recruitment design language. Constructed an interactive recruiter workspace across 11 primary application routes (/overview, /new-analysis, /requirements-review, /analysis, /candidates/[id], /reports, etc.), implementing candidate comparison grids, interactive weight sliders, and citation inspection modals.
- Step 2: Backend Architecture & Core NLP Pipeline (Milestone 2)
Engineered the stateless FastAPI backend. Implemented in-memory PDF extraction via PyMuPDF without OCR to avoid heavy binary dependencies. Structured custom token preservation in spaCy for punctuated technologies (e.g., C++, C#, .NET), and enforced strict JD parsing boundaries ensuring optional preferred skills are never conflated with mandatory requirements.
- Step 3: End-to-End Integration & Explainability Engine (Milestone 3)
Connected Next.js client services to FastAPI endpoints (/analyze/match, /analyze/rank). Built the explainability engine to extract exact sentence context and page numbers for verified skills. Enforced strict non-inferential absence framing: unmatched skills are explicitly tagged 'Not found in resume' rather than claiming a candidate lacks technical capability.

Step 4: Objective Evaluation Harness & Benchmark Dataset (Milestone 4)
Engineered an automated benchmark harness under backend/evaluation/. Formulated 10 diverse synthetic resumes and 4 industry job requisitions completely free of demographic attributes or personal data. Established expert ground-truth annotations and implemented standard IR metrics: Micro/Macro Precision, Recall, F1 score, and Mean NDCG@K ranking quality.

Step 5: Algorithm Tuning, Boundary Safeguards & Scoring Validation (Milestone 5)
Investigated and eliminated five documented skill false-positives. Implemented dual-boundary regex patterns for C and C++ to prevent single-letter English collisions, added case-sensitive deterministic matching for Go, and purged bare aliases ('node', 'rest'). Validated the 70/30 scoring formula through empirical experiments comparing 50/50, 60/40, 70/30, and 80/20.

Step 6: Production Hardening & Input Safeguards (Milestone 6)
Hardened API endpoints against payload exhaustion and invalid data: (1) 10MB upload limit returning HTTP 413, (2) %PDF- magic byte verification rejecting renamed non-PDFs (HTTP 400), (3) 50-file batch capping on /rank (HTTP 400), (4) candidate failure isolation allowing valid resumes to rank even if corrupt files exist in the batch, and (5) exception sanitization.

EMPIRICAL BENCHMARK & REGRESSION RESULTS

Evaluation Dimension	Benchmark Metric / Result	Evaluation Scope & Notes
Skill Extraction Accuracy	100% Precision 100% Recall 100% F1	Controlled 10-resume benchmark corpus
Ranking Quality (NDCG@5)	1.0000 Mean NDCG@5	Strict correlation with expert ground-truth
Regression Baseline Anchor	Score: 40.76 Sim: 15.37% Cov: 100.0%	candidate_alpha.pdf vs ml_engineer.txt
Automated Test Verification	59 Passed, 1 Warning (Pytest in 9.5s)	Full API, boundary, and scoring coverage

5. CONCLUSION & FUTURE SCOPE

The development of ResumeIQ demonstrates that deterministic, rule-assisted NLP heuristics provide a far more robust, transparent, and operationally reliable foundation for recruitment screening than unconstrained generative AI models. By grounding match scores in verifiable textual evidence, sublinear TF-IDF term frequency, and controlled skill taxonomy matching, ResumeIQ delivers complete auditability while preventing hallucinations and algorithmic drift.

All engineering deliverables across Milestones 1 through 6 have been fully verified and frozen: 59 automated test cases passing, 13 frontend routes compiling cleanly, and the candidate_alpha regression baseline strictly preserved at 40.76. Future scope includes optional OCR plugins for scanned paper archives, enterprise applicant tracking system (ATS) integrations, and expanded multi-lingual taxonomy mappings.

VERIFICATION STATEMENT